FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	**Alvin Christian S. Panio**
Section:	**AN43-BSCSSE**
Professor:	**Crisola Tan**
Date:	**October 25, 2026**

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
    (Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF-THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule
3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]

```
Rule-Based Reasoning Actions:
- Turn on the aircon
- Bring umbrella
```

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
2. Which is more flexible in solving unstructured problems?
3. Which is better for situations with continuously changing data?
4. Examples of real-world applications for both.

Your Code - Laboratory Tasks ⇩

Parts 1 and 2 Knowledge base and rule based reasoning

Use case: an illustrative weather activity assistant recommends practical actions from current conditions. Both reasoning methods receive the same five facts. The observations and past cases are educational examples, not live weather or personal records.

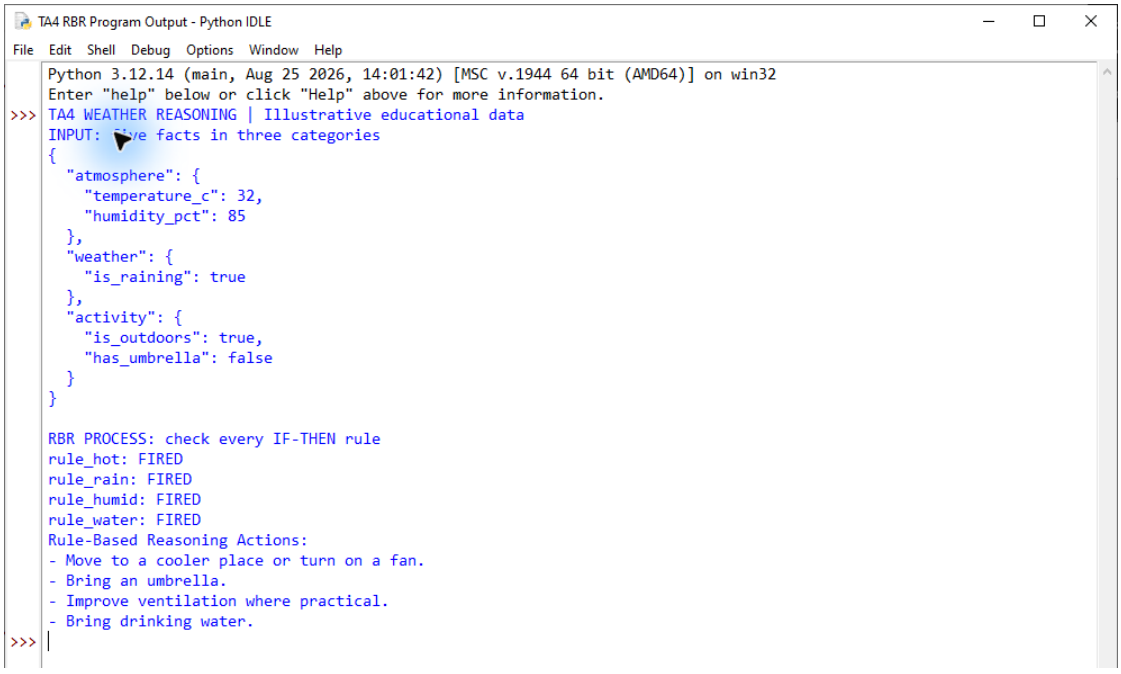
Task 1A The dictionary groups five facts into three categories: atmosphere (temperature and humidity), weather (rain), and activity (outdoor status and umbrella availability). Numeric values use degrees Celsius and percent; Boolean values record presence or status.

```
facts = {
    "atmosphere": {"temperature_c": 32, "humidity_pct": 85},
    "weather": {"is_raining": True},
    "activity": {"is_outdoors": True, "has_umbrella": False}
}
```

Tasks 2A and 2B Four Python functions express the rules below. The inference engine flattens the categorized facts into working memory, checks every rule, records which rules fire, and collects all resulting actions without duplicates. One pass is sufficient because these rules output actions rather than new facts.

R1: IF temperature > 30, THEN move to a cooler place or turn on a fan.
R2: IF raining AND outdoors AND no umbrella, THEN bring an umbrella.
R3: IF humidity >= 80, THEN improve ventilation where practical.
R4: IF outdoors AND temperature >= 30, THEN bring drinking water.

Task 2C Actual RBR output



All four conditions match the example input. When no rule matches, the program explicitly reports that no action was inferred. This is an actual screenshot of the program running in Python IDLE.

Part 3 Case based reasoning

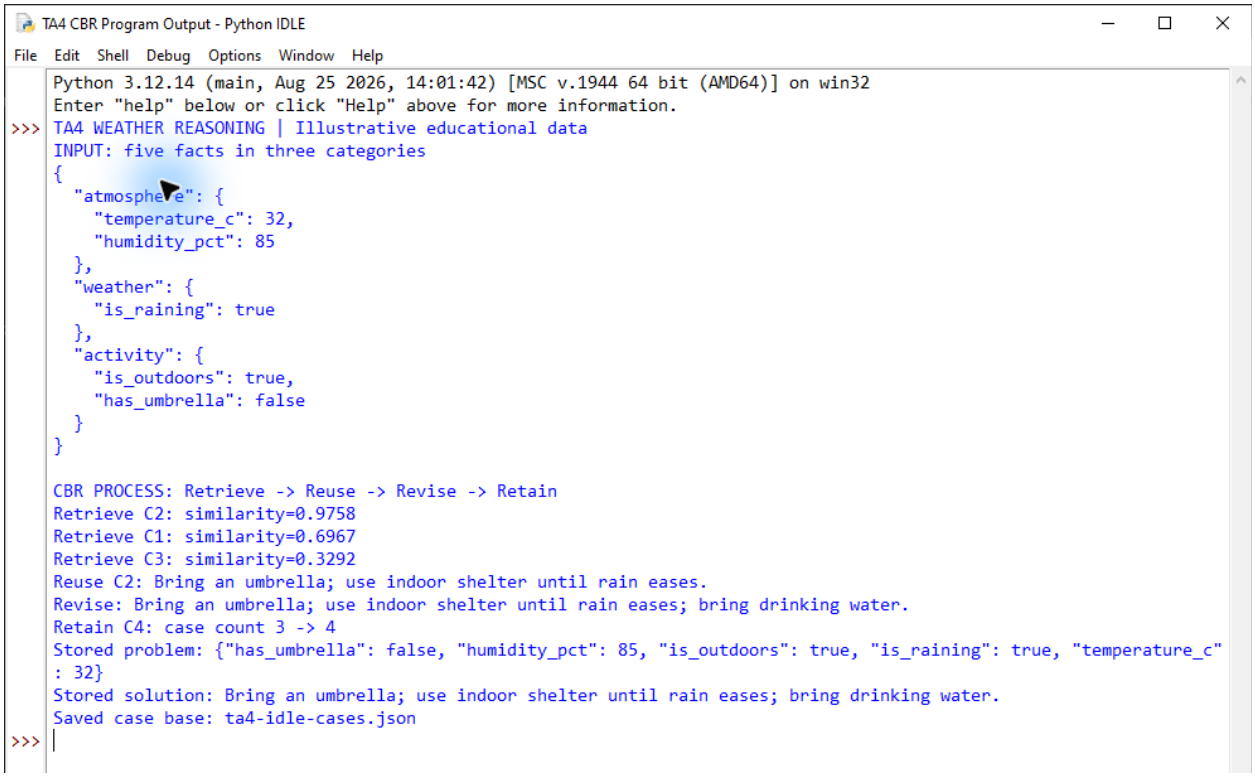
Task 3A Three illustrative past cases pair the same five problem features with a solution. Feature order below is temperature, humidity, raining, outdoors, and has umbrella.

C1: (34, 55, False, True, False) -> Use shade and bring drinking water.
C2: (28, 90, True, True, False) -> Bring an umbrella; use indoor shelter until rain eases.
C3: (24, 60, False, False, True) -> Continue indoor activity; no extra weather action needed.

Retrieve Each case receives a weighted similarity score from 0 to 1. Numeric similarity is 1 - abs(query - case) / range; the temperature range is 60 (-10 to 50) and humidity range is 100 (0 to 100). Boolean similarity is 1 for an exact match and 0 otherwise. These declared ranges and weights are design choices for this small example.

Weights: temperature 0.25, humidity 0.15, rain 0.25, outdoors 0.20, umbrella 0.15. The score is the weighted mean. C2 scores 0.25(56/60) + 0.15(95/100) + 0.25 + 0.20 + 0.15 = 0.9758. It ranks above C1 (0.6967) and C3 (0.3292). Ties retain original case order.

Tasks 3C to 3E Reuse selects C2's solution. Revise accepts a manual replacement or keeps the solution when the entry is blank. The demonstrated replacement adds drinking water for the hot outdoor conditions. Retain stores the current features and revised solution as C4. With --case-file, the learned case is saved as JSON and can be retrieved on a later run.



Actual screenshot of the CBR program running in Python IDLE. The displayed revision is a demonstration, not evidence of a tested real-world outcome. Retaining reviewed experiences supports learning; unreviewed solutions may preserve mistakes.

Part 4 Comparative reflection

For this implementation, RBR is easier to implement because each condition maps directly to an action, and one loop checks four readable functions. CBR is more flexible for partially matching or less structured problems because a new problem can reuse a similar experience even without an exact rule, although useful features and a suitable similarity measure are still necessary. CBR can adapt to new problem patterns by retaining reviewed cases; RBR remains effective when input values change but the decision rules stay valid. Neither method automatically handles every kind of changing data: rules may need revision, while cases can become outdated or incorrect. Practical applications include RBR for equipment threshold alarms or eligibility checks, and CBR for retrieving previous help-desk resolutions or equipment repair cases.

Observed comparison

On the identical hot, humid, rainy outdoor input, RBR infers four separate actions from current facts. CBR first reuses the closest rainy-weather experience and then adds drinking water through revision. RBR explains its answer with fired rule names; CBR explains its starting answer with the selected case and similarity score. The retained C4 is an exact feature match on the next run, so its similarity becomes 1.0000.

Execution and verification

Run with Python 3.10 or later; only the standard library is required. The default run prints input, rule checks, actions, and all four CBR stages. Optional interactive revision supports keyboard input. A fresh case filename starts with three seed cases; an existing file continues learning.

```
python ta4_weather_reasoning.py
python ta4_weather_reasoning.py --mode rbr
python ta4_weather_reasoning.py --mode cbr --interactive
python ta4_weather_reasoning.py --case-file learned_cases.json
python -m unittest -v test_ta4
```

Verification result: all 9 behavioral tests passed. The tests check the five-fact/three-category structure, all matching rules, threshold and no-match behavior, the existing-umbrella condition, exact/opposite similarity and expected ranking, deterministic ties and an empty case base, manual/unchanged revision and copied retention, invalid input rejection, and saved learning retrieved in a second process.

The tests confirmed that a fresh run retains C4 (3 -> 4 cases), and a second run using the same file reuses C4 and retains C5 (4 -> 5). This small demonstration stores every completed experience; a larger application should review outcomes, detect duplicates, and retire stale cases. The complete laboratory program follows; the separate test_ta4.py file contains the executable checks.

Complete Python program

```
"""TA4: illustrative weather knowledge, rules, and a complete CBR cycle.

Python 3.10+, standard library only. No live weather or personal records.
Run: python ta4_weather_reasoning.py
Use --interactive to revise the retrieved solution yourself.
Use --case-file learned_cases.json to save learning across runs.
"""

import argparse
import json
from copy import deepcopy
from pathlib import Path


# Five facts, grouped explicitly into three knowledge categories.
FACTS = {
    "atmosphere": {"temperature_c": 32, "humidity_pct": 85},
    "weather": {"is_raining": True},
    "activity": {"is_outdoors": True, "has_umbrella": False},
}
WEIGHTS = dict(temperature_c=0.25, humidity_pct=0.15,
               is_raining=0.25, is_outdoors=0.20, has_umbrella=0.15)
RANGES = {"temperature_c": (-10, 50), "humidity_pct": (0, 100)}
SEED_CASES = [
    {"id": "C1", "problem": dict(temperature_c=34, humidity_pct=55,
     is_raining=False, is_outdoors=True, has_umbrella=False),
     "solution": "Use shade and bring drinking water."},
    {"id": "C2", "problem": dict(temperature_c=28, humidity_pct=90,
     is_raining=True, is_outdoors=True, has_umbrella=False),
     "solution": "Bring an umbrella; use indoor shelter until rain eases."},
    {"id": "C3", "problem": dict(temperature_c=24, humidity_pct=60,
     is_raining=False, is_outdoors=False, has_umbrella=True),
     "solution": "Continue indoor activity; no extra weather action needed."},
]


def flatten(facts):
    """Expose the three categories as five comparable feature values."""
    features = {key: value for group in facts.values()
                for key, value in group.items()}
    validate(features)
    return features


def validate(features):
    """Reject missing, extra, mistyped, or out-of-range feature values."""
    if set(features) != set(WEIGHTS):
        raise ValueError("Exactly the five declared features are required.")
    for key, value in features.items():
```

Complete Python program continued

```
        if key in RANGES:
            low, high = RANGES[key]
            if type(value) not in (int, float) or not low <= value <= high:
                raise ValueError(f"{key} must be a number in [{low}, {high}].")
        elif type(value) is not bool:
            raise ValueError(f"{key} must be True or False.")


def rule_hot(f):
    if f["temperature_c"] > 30:
        return "Move to a cooler place or turn on a fan."


def rule_rain(f):
    if f["is_raining"] and f["is_outdoors"] and not f["has_umbrella"]:
        return "Bring an umbrella."


def rule_humid(f):
    if f["humidity_pct"] >= 80:
        return "Improve ventilation where practical."


def rule_water(f):
    if f["is_outdoors"] and f["temperature_c"] >= 30:
        return "Bring drinking water."


RULES = [rule_hot, rule_rain, rule_humid, rule_water]


def infer(features):
    """A single forward pass is enough: actions do not create new facts."""
    validate(features)
    actions, trace = [], []
    for rule in RULES:
        action = rule(features)
        trace.append((rule.__name__, action is not None))
        if action and action not in actions:
            actions.append(action)
    return actions, trace


def similarity(query, problem):
    """Weighted mean of normalized numeric and exact Boolean matches."""
    validate(query)
    validate(problem)
    total = 0.0
    for key, weight in WEIGHTS.items():
```

Complete Python program continued

```
        if key in RANGES:
            low, high = RANGES[key]
            score = 1 - abs(query[key] - problem[key]) / (high - low)
        else:
            score = float(query[key] == problem[key])
        total += weight * score
    return total / sum(WEIGHTS.values())


def retrieve(query, cases):
    """Return all ranked cases; insertion order breaks equal-score ties."""
    if not cases:
        raise ValueError("Cannot retrieve from an empty case base.")
    return sorted(((similarity(query, c["problem"]), c) for c in cases),
                  key=lambda pair: pair[0], reverse=True)


def revise(solution, manual_revision=""):
    """An empty revision explicitly keeps the retrieved solution."""
    return manual_revision.strip() or solution


def retain(query, solution, cases):
    """Store the revised experience without aliasing the query object."""
    validate(query)
    if not solution.strip():
        raise ValueError("A retained solution must not be empty.")
    used = {case["id"] for case in cases}
    number = 1
    while f"C{number}" in used:
        number += 1
    new_case = {"id": f"C{number}", "problem": deepcopy(query),
                "solution": solution.strip()}
    cases.append(new_case)
    return new_case


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--mode", choices=["all", "rbr", "cbr"], default="all")
    parser.add_argument("--interactive", action="store_true")
    parser.add_argument("--revision", default="")
    parser.add_argument("--case-file", type=Path)
    args = parser.parse_args()
    features = flatten(FACTS)
    print("TA4 WEATHER REASONING | Illustrative educational data")
    print("INPUT: five facts in three categories")
    print(json.dumps(FACTS, indent=2))
    if args.mode in ("all", "rbr"):
```

Complete Python program continued

```
        actions, trace = infer(features)
        print("\nRBR PROCESS: check every IF-THEN rule")
        for name, fired in trace:
            print(f"{name}: {'FIRED' if fired else 'not fired'}")
        print("Rule-Based Reasoning Actions:")
        for action in actions:
            print("-", action)
        if not actions:
            print("- No rule matched; no action inferred.")
    if args.mode in ("all", "cbr"):
        cases = deepcopy(SEED_CASES)
        if args.case_file and args.case_file.exists():
            cases = json.loads(args.case_file.read_text(encoding="utf-8"))
        ranked = retrieve(features, cases)
        print("\nCBR PROCESS: Retrieve -> Reuse -> Revise -> Retain")
        for score, case in ranked:
            print(f"Retrieve {case['id']}: similarity={score:.4f}")
        score, chosen = ranked[0]
        solution = chosen["solution"]  # Reuse the best matching experience.
        print(f"Reuse {chosen['id']}: {solution}")
        revision = args.revision
        if args.interactive:
            revision = input("Revised solution (Enter keeps it unchanged): ")
        solution = revise(solution, revision)
        print("Revise:", solution)
        before = len(cases)
        added = retain(features, solution, cases)
        print(f"Retain {added['id']}: case count {before} -> {len(cases)}")
        print("Stored problem:", json.dumps(added["problem"], sort_keys=True))
        print("Stored solution:", added["solution"])
        if args.case_file:
            args.case_file.write_text(json.dumps(cases, indent=2) + "\n",
                                      encoding="utf-8")
            print("Saved case base:", args.case_file.name)
        else:
            print("Learning retained in memory for this run.")


if __name__ == "__main__":
    main()
```